

# 9

# CAMPAMENTO DE PROGRAMACIÓN COMPETITIVA

ENERO 2019



The background features an abstract composition of various-sized 3D rectangular blocks in shades of brown and tan. These blocks are arranged in a way that creates a sense of depth and perspective, with some blocks appearing to be stacked or overlapping. The lighting is soft, casting subtle shadows that enhance the three-dimensional effect.

## **9. Estructuras de Datos II**

# Día 9. Estructuras de Datos II



Oscar Gauss Carvajal Yucra

**Carrera de Informática**  
**Facultad de Ciencias Puras y Naturales**



# Estructuras de Datos II



1 Segment Tree

2 Sparse Table



1

# Segment Tree



**UMSA**  
*la mejor*

# Introducción



Consideremos el siguiente problema: Tenemos una array  $A$   $[0 \dots n-1]$ . Nos gustaría:

- Calcular el mínimo elemento de los elementos de  $i$  a  $j$ ,  $query(i, j)$ .
- Modificar el valor de un elemento específico del array  $A[i] = v$  donde  $0 \leq i \leq n - 1$ ,  $update(i, v)$ .

Una solución simple es ejecutar un bucle de  $i$  a  $j$  y calcular el mínimo de los elementos. Para actualizar un valor, simplemente  $A[i] = x$ . La primera operación toma tiempo  $O(n)$  y la segunda operación toma tiempo  $O(1)$ .

Que pasa si tenemos muchísimas  $n$ , preguntas del tipo  $query(i, j)$ , nos tomaría  $O(n^2)$  en responder, por lo que nuestro array debería tener 1000 elementos como máximo



# Introducción



Consideremos el siguiente problema: Tenemos una array  $A$   $[0 \dots n-1]$ . Nos gustaría:

- Calcular el mínimo elemento de los elementos de  $i$  a  $j$ ,  $query(i, j)$ .
- Modificar el valor de un elemento específico del array  $A[i] = v$  donde  $0 \leq i \leq n - 1$ ,  $update(i, v)$ .

Una solución simple es ejecutar un bucle de  $i$  a  $j$  y calcular el mínimo de los elementos. Para actualizar un valor, simplemente  $A[i] = x$ . La primera operación toma tiempo  $O(n)$  y la segunda operación toma tiempo  $O(1)$ .

Que pasa si tenemos muchísimas  $n$ , preguntas del tipo  $query(i, j)$ , nos tomaría  $O(n^2)$  en responder, por lo que nuestro array debería tener 1000 elementos como máximo



# Introducción



Consideremos el siguiente problema: Tenemos una array  $A$   $[0 \dots n-1]$ . Nos gustaría:

- Calcula el minimo elemento de los elementos de  $i$  a  $j$ ,  $query(i, j)$ .
- Modificar el valor de un elemento específico del array  $A[i] = v$  donde  $0 \leq i \leq n - 1$ ,  $update(i, v)$ .

Una solución simple es ejecutar un bucle de  $i$  a  $j$  y calcular el minimo de los elementos. Para actualizar un valor, simplemente  $A[i] = x$ . La primera operación toma tiempo  $O(n)$  y la segunda operación toma tiempo  $O(1)$ .

Que pasa si tenemos muchísimas  $n$ , preguntas del tipo  $query(i, j)$ , nos tomaría  $O(n^2)$  en responder, por lo que nuestro array debería tener 1000 elementos como máximo





# Introducción



Consideremos el siguiente problema: Tenemos una array  $A$   $[0 \dots n-1]$ . Nos gustaría:

- Calcula el minimo elemento de los elementos de  $i$  a  $j$ ,  $query(i, j)$ .
- Modificar el valor de un elemento específico del array  $A[i] = v$  donde  $0 \leq i \leq n - 1$ ,  $update(i, v)$ .

Una solución simple es ejecutar un bucle de  $i$  a  $j$  y calcular el minimo de los elementos. Para actualizar un valor, simplemente  $A[i] = x$ . La primera operación toma tiempo  $O(n)$  y la segunda operación toma tiempo  $O(1)$ .

Que pasa si tenemos muchísimas  $n$ , preguntas del tipo  $query(i, j)$ , nos tomaría  $O(n^2)$  en responder, por lo que nuestro array debería tener 1000 elementos como máximo



# Introducción



Consideremos el siguiente problema: Tenemos una array  $A$   $[0 \dots n-1]$ . Nos gustaría:

- Calcula el minimo elemento de los elementos de  $i$  a  $j$ ,  $query(i, j)$ .
- Modificar el valor de un elemento específico del array  $A[i] = v$  donde  $0 \leq i \leq n - 1$ ,  $update(i, v)$ .

Una solución simple es ejecutar un bucle de  $i$  a  $j$  y calcular el minimo de los elementos. Para actualizar un valor, simplemente  $A[i] = x$ . La primera operación toma tiempo  $O(n)$  y la segunda operación toma tiempo  $O(1)$ .

Que pasa si tenemos muchísimas  $n$ , preguntas del tipo  $query(i, j)$ , nos tomaría  $O(n^2)$  en responder, por lo que nuestro array debería tener 1000 elementos como máximo



# Introducción



Consideremos el siguiente problema: Tenemos una array  $A$   $[0 \dots n-1]$ . Nos gustaría:

- Calcula el minimo elemento de los elementos de  $i$  a  $j$ ,  $query(i, j)$ .
- Modificar el valor de un elemento específico del array  $A[i] = v$  donde  $0 \leq i \leq n - 1$ ,  $update(i, v)$ .

Una solución simple es ejecutar un bucle de  $i$  a  $j$  y calcular el minimo de los elementos. Para actualizar un valor, simplemente  $A[i] = x$ . La primera operación toma tiempo  $O(n)$  y la segunda operación toma tiempo  $O(1)$ .

Que pasa si tenemos muchísimas  $n$ , preguntas del tipo  $query(i, j)$ , nos tomaría  $O(n^2)$  en responder, por lo que nuestro array debería tener 1000 elementos como máximo



# Introducción



Consideremos el siguiente problema: Tenemos una array  $A$   $[0 \dots n-1]$ . Nos gustaría:

- Calcula el minimo elemento de los elementos de  $i$  a  $j$ ,  $query(i, j)$ .
- Modificar el valor de un elemento específico del array  $A[i] = v$  donde  $0 \leq i \leq n - 1$ ,  $update(i, v)$ .

Una solución simple es ejecutar un bucle de  $i$  a  $j$  y calcular el minimo de los elementos. Para actualizar un valor, simplemente  $A[i] = x$ . La primera operación toma tiempo  $O(n)$  y la segunda operación toma tiempo  $O(1)$ .

Que pasa si tenemos muchísimas  $n$ , preguntas del tipo  $query(i, j)$ , nos tomaría  $O(n^2)$  en responder, por lo que nuestro array debería tener 1000 elementos como máximo



# ¿Podríamos realizar las operaciones de consulta y actualización en tiempo $O(\log n)$ ?



Sip, con Segment Tree

Segment Tree realiza ambas operaciones en tiempo  $O(\log n)$ .

¿Como?



# ¿Podríamos realizar las operaciones de consulta y actualización en tiempo $O(\log n)$ ?



Sip, con Segment Tree

Segment Tree realiza ambas operaciones en tiempo  $O(\log n)$ .

¿Como?



# ¿Podríamos realizar las operaciones de consulta y actualización en tiempo $O(\log n)$ ?



Sip, con Segment Tree

Segment Tree realiza ambas operaciones en tiempo  $O(\log n)$ .

¿Como?



# ¿Podríamos realizar las operaciones de consulta y actualización en tiempo $O(\log n)$ ?



Sip, con Segment Tree

Segment Tree realiza ambas operaciones en tiempo  $O(\log n)$ .

¿Como?





# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...



# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...



# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...



# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...



# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...



# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...



# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...



# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...





# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...



# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...



# Segment Tree



Segment Tree funciona excelente con operaciones y funciones asociativas.

Como por ejemplo con:

- Suma  $a + b$
- Multiplicación  $a * b$
- And  $a \& b$
- Or  $a | b$
- Mínimo  $\min(a, b)$
- Máximo  $\max(a, b)$
- Máximo Común Divisor  $\text{__gcd}(a, b)$
- ...



The background consists of several overlapping, three-dimensional rectangular blocks in various shades of brown and tan. The blocks are arranged in a way that creates a sense of depth and perspective, with some blocks appearing to be in front of others. The lighting is soft, creating subtle shadows and highlights on the edges of the blocks.

**Veamos como funciona...**

2

# Sparse Table



**UMSA**  
*la mejor*

# Introducción



Consideremos el anterior problema pero ahora no nos interesa cambiar valores en el array pero si necesitamos encontrar el mínimo elemento de los elementos  $i$  a  $j$  en tiempo constante!!!... Sparse Table logra realizar esta operación en tiempo constante ¿Como?



# Introducción



Consideremos el anterior problema pero ahora no nos interesa cambiar valores en el array pero si necesitamos encontrar el mínimo elemento de los elementos  $i$  a  $j$  en tiempo constante!!!!...  
Sparse Table logra realizar esta operación en tiempo constante  
¿Como?



# Introducción



Consideremos el anterior problema pero ahora no nos interesa cambiar valores en el array pero si necesitamos encontrar el mínimo elemento de los elementos  $i$  a  $j$  en tiempo constante!!!!... Sparse Table logra realizar esta operación en tiempo constante  
¿Como?





# Introducción



Consideremos el anterior problema pero ahora no nos interesa cambiar valores en el array pero si necesitamos encontrar el mínimo elemento de los elementos  $i$  a  $j$  en tiempo constante!!!!... Sparse Table logra realizar esta operación en tiempo constante ¿Como?



The background features an abstract composition of various-sized 3D rectangular blocks. These blocks are rendered in a palette of earthy browns, from light tan to dark chocolate and near-black tones. They are arranged in a way that creates a sense of depth and perspective, with some blocks appearing to be stacked or placed on top of others. The lighting is soft, casting subtle shadows that emphasize the three-dimensional nature of the shapes.

**Veamos como funciona...**

The background features an abstract composition of 3D rectangular blocks in various shades of brown and tan. The blocks are arranged in a way that creates a sense of depth and perspective, with some blocks appearing to be stacked or overlapping. The lighting is soft, casting subtle shadows that enhance the three-dimensional effect. The overall aesthetic is clean and modern.

**¿Preguntas?**

The background features a series of overlapping, three-dimensional rectangular blocks. The blocks are rendered in various shades of brown, from dark chocolate to light tan, creating a sense of depth and perspective. The arrangement is somewhat chaotic, with blocks of different sizes and orientations. A horizontal band of light gray or off-white color runs across the middle of the image, serving as a backdrop for the text.

**Gracias!**